

Solana Esports Platform

Monetized Matchmaking, MPC Wallets & Prize Distribution

Production-Grade Implementation

Agenda

1. **Security Concepts** - Understanding Threats & Mitigations
2. **System Overview** - Architecture & Components
3. **Multi-Token Support** - SOL, USDC, USDT, PYUSD Integration
4. **Service Layer Architecture** - Core Services Deep Dive
5. **MPC Wallet Integration** - Secure Player Wallets
6. **Matchmaking System** - Entry Fees & Escrow
7. **Prize Distribution** - Calculations & Strategies
8. **Tournament Management** - Bracket Generation
9. **Security Implementation** - Production Considerations



Security Concepts: Overview

Threats We Address

Threat	Impact
Collusion	Unfair outcomes
Sybil Attacks	Pool manipulation
Fund Theft	Asset loss
Prize Manipulation	Incorrect payouts
Result Tampering	False winners

Our Mitigations

Solution	Protects Against
MPC Wallets	Fund theft
Escrow System	Payment disputes
Entry Validation	Sybil attacks
Deterministic Prizes	Manipulation
Audit Logs	Tampering

What is Collusion?

Definition

Collusion = Secret cooperation between players to gain unfair advantage at the expense of others.

Gaming Examples

- **Match Fixing:** Players agree on outcome before match starts
- **Chip Dumping:** Intentionally losing to transfer prize pool
- **Ghosting:** Sharing opponent's screen/position info
- **Win Trading:** Alternating wins to farm rewards

Why It Matters

- Destroys competitive integrity
- Defrauds legitimate players
- Legal liability for platform
- Regulatory scrutiny

Our Mitigations

- ✓ Random matchmaking queues
- ✓ Anti-patterns in match history
- ✓ Prize distribution audit trails
- ✓ Platform co-signing (MPC)



What is a Sybil Attack?

Definition

Sybil Attack = Creating multiple fake identities to gain disproportionate influence or rewards.

Gaming Examples

- **Multi-accounting:** One player, many accounts
- **Prize Pool Stuffing:** Fill match with own accounts
- **Guaranteed Wins:** Play against yourself
- **Bonus Abuse:** Claim rewards multiple times

Why It Matters

- Drains prize pools illegitimately
- Destroys fair competition
- Inflates platform costs
- Undermines trust

Our Mitigations

- ✓ Entry fee requirements (economic barrier)
- ✓ MPC wallet creation limits
- ✓ IP/device fingerprinting (external)
- ✓ Behavioral analysis patterns



What is Escrow?

Definition

Escrow = Neutral third-party holds funds until conditions are met, then releases to appropriate party.

How It Works



Why It Matters

- No trust required between players
- Platform can't steal funds
- Automatic settlement on result
- Dispute resolution possible

Our Implementation

- ✓ On-chain escrow accounts (PDA-style)
- ✓ Locked until match complete
- ✓ Deterministic release conditions
- ✓ Refund on cancellation/timeout



What is MPC (Multi-Party Computation)?

Definition

MPC = Cryptographic technique where multiple parties jointly compute a result (signature) without revealing their individual inputs (key shares).

2-of-3 Threshold Scheme

KEY SHARES
Share 1: Player
Share 2: Platform
Share 3: Recovery
Sign = Any 2 shares

Why It Matters

- **No single point of failure** - one compromised key ≠ fund loss
- **No seed phrases** - better UX for players
- **Platform co-signs** - prevents fraud withdrawals
- **Recovery possible** - lost device ≠ lost funds

Security Properties

- ✓ Player + Platform = normal operation
- ✓ Player + Recovery = device lost
- ✓ Platform alone = cannot steal funds



What is Prize Manipulation?

Definition

Prize Manipulation = Tampering with prize calculations or distribution to benefit certain parties unfairly.

Attack Vectors

- **Floating point errors:** Rounding exploits
- **Percentage overflow:** >100% claimed
- **Fee miscalculation:** Platform takes extra
- **Missing distributions:** Prizes disappear

Why It Matters

- Direct financial loss to players
- Legal liability for platform
- Destroys trust instantly
- Regulatory violations

Our Mitigations

- ✓ **BigInt only** - no floating point
- ✓ **Basis points** - 100 = 1% precision
- ✓ **Pre-defined strategies** - auditable
- ✓ **Sum validation** - must equal 100%



What is Entry Fee Validation?

Definition

Entry Fee Validation = Ensuring each participant pays the exact required fee in the correct token before joining.

Attack Vectors Without It

- Pay 0, win full prize pool
- Pay in wrong token (worthless)
- Partial payment exploitation
- Cross-token arbitrage

Why It Matters

- Prize pool integrity
- Fair competition (equal stake)
- Token consistency
- Economic security

Our Implementation

- ✓ Token-specific fee ranges
- ✓ Exact amount verification
- ✓ Balance check before lock
- ✓ Atomic lock → join operation



What is Deterministic Addressing?

Definition

Deterministic Addressing = Deriving account addresses from fixed inputs so the same inputs always produce the same address (like Solana PDAs).

Formula

```
escrow_addr = hash(  
  source_type + source_id  
)  
// Always same for Match #123
```

Why It Matters

- **Verifiable**: Anyone can compute address
- **No guessing**: Can't find random escrow
- **No collisions**: Unique per match/tournament
- **Auditable**: Trace funds to source

Our Implementation

- ✓ SHA-256 hash of source type + ID
- ✓ Match escrow = `MATCH:match_xxx`
- ✓ Tournament = `TOURNAMENT:tour_xxx`
- ✓ Cannot be manipulated



What is Rate Limiting?

Definition

Rate Limiting = Restricting how often an action can be performed within a time window.

Gaming Context

Action	Limit	Purpose
Withdrawals	3/day	Theft delay
Match joins	10/hour	Anti-bot
Wallet create	1/user	Anti-sybil

Why It Matters

- Slows down attackers
- Time for fraud detection
- Protects against automation
- Compliance (suspicious activity)

Our Implementation

- ✓ Daily withdrawal limits
- ✓ Cooldown periods
- ✓ Max active matches per player
- ✓ Tournament registration windows

What is an Audit Trail?

Definition

Audit Trail = Immutable record of all actions, state changes, and transactions for accountability and forensics.

What We Log

Event	Data Captured
Wallet create	Player, MPC ID, time
Match join	Player, fee, escrow ref
Result submit	Winners, proof, signer
Prize distribute	Amounts, recipients, tx

Why It Matters

- **Dispute resolution** - prove what happened
- **Fraud detection** - identify patterns
- **Compliance** - regulatory requirements
- **Debugging** - trace issues

Our Implementation

- ✓ All entities have createdAt/updatedAt
- ✓ Transaction history per wallet
- ✓ Escrow state transitions logged
- ✓ Blockchain signatures stored

System Architecture Overview





Multi-Token Support

Why Multi-Token?

Benefit	Description
Price Stability	Fixed USD value during matches
User Choice	Preferred token support
Compliance	USD values for tax reporting
Low Volatility	Stable tournament prizes

Supported Tokens

Token	Type	Decimals
SOL	Native	9
USDC	Stablecoin	6
USDT	Stablecoin	6
PYUSD	Stablecoin	6
WSOL	Wrapped	9



Token Configuration Architecture

Token Enum

```
export enum SupportedToken {  
  SOL = 'SOL',    // 9 decimals  
  USDC = 'USDC',  // 6 decimals  
  USDT = 'USDT',  // 6 decimals  
  PYUSD = 'PYUSD', // 6 decimals  
  WSOL = 'WSOL',  // 9 decimals  
}
```

Config Example (USDC)

```
[SupportedToken.USDC]: {  
  decimals: 6,  
  mintAddress: {  
    mainnet: 'EPjFWdd5...Dt1v',  
    devnet: '4zMMC9sr...ncDU',  
  },  
  isStablecoin: true,  
  minEntryFee: '100000',  
  maxEntryFee: '1000000000000',  
}
```



Multi-Token Wallet Architecture

Per-Token Balances

```
WALLET (MPC 2-of-3)
├ SOL: 1.5 avail, 0.5 lock
├ USDC: 150 avail, 50 lock
├ USDT: 75 avail, 0 lock
└ PYUSD: 0 avail, 0 lock
```

Balance Isolation

Match Requires	Can Use
10 USDC	USDC only
1 SOL	SOL only
Mixed	✗ Never

⚠ Prevents cross-token exploits

Token-Aware Escrow Flow

Match Creation

```
POST /esports/matches
{ tokenType: "USDC",
  entryFee: "5000000" }

// Validates: 0.10 - 100k USDC
// Creates: USDC escrow + ATA
```

Token Enforcement

```
Match: USDC
├ Player 1: USDC ✓
├ Player 2: USDC ✓
└ Player 3: SOL ✗
```

 **Token immutable after creation**



Decimal Handling

Token Decimals

Token	Dec	1.00 =
SOL	9	1000000000
USDC/USDT	6	1000000

```
toBaseUnits('USDC', '10.50')  
// → '10500000'
```



Use BigInt!

```
// ✓ CORRECT  
const total = BigInt(fee)  
              * BigInt(count);  
  
// ✗ WRONG!  
const bad = parseFloat(fee)  
            * count;
```

Stablecoin Entry Fee Example

\$5 USDC Match

```
const match = await createMatch({  
  gameType: BATTLE_ROYALE,  
  tokenType: USDC,  
  entryFee: '5000000',  
  maxPlayers: 100,  
  prizeStrategy: TOP_3_SPLIT,  
});
```

Prize Pool

100 × 5 USDC = 500 USDC
Platform (5%): -25 USDC

Available: 475 USDC

1st: 285 (60%)
2nd: 142.5 (30%)
3rd: 47.5 (10%)



Token Security Checklist

✓ Mint Address Verification

- Hardcoded official addresses
- No dynamic token addition
- Separate mainnet/devnet

✓ Decimal Handling

- Always use BigInt
- Store base units only
- UI conversion at display

✓ ATA Management

- Create before deposit
- Validate ownership
- Handle rent-exempt

✓ Transfer Validation

- Verify mint matches
- Check balance first
- Validate recipient ATA

✓ Balance Isolation

- Per-token tracking
- No cross-token spending
- Enforced at match level

✓ Audit Trail

- Log token types
- Track mint addresses
- Record ATA derivations



Token Implementation Do's and Don'ts

✓ DO's

Practice	Reason
Hardcode mint addresses	Prevents substitution attacks
Validate on every transfer	Token type consistency
Use BigInt everywhere	Prevents precision loss
Store base units only	Simplifies calculations
Derive ATAs deterministically	Correct account targeting
Log token type	Aids debugging/auditing

✗ DON'Ts

Anti-Pattern	Risk
Arbitrary mint addresses	Substitution attacks
Mix token types	Wrong distribution
Use floating point	Precision loss
Skip mint validation	Wrong token accepted
Cross-token operations	Exploitation
Store display amounts	Decimal errors

Querying by Token Type

Filter Matches/Tournaments

```
// Stablecoin matches only
GET /esports/matches
  ?stablecoinsOnly=true

// USDC tournaments
GET /esports/tournaments
  ?tokenType=USDC
  &status=REGISTRATION_OPEN
```

Response Fields

```
interface MatchResponse {
  matchId: string;
  tokenType: SupportedToken;
  tokenMint: string;
  entryFee: string;
  prizePool: string;
}
```

Service Layer Architecture

Five Core Services



PlayerWalletService

MPC-Secured Wallet Management

```
@Injectable()
export class PlayerWalletService {
  // Creates MPC wallet with 2-of-3 threshold scheme
  async createWallet(request: CreatePlayerWalletRequest) {
    const mpcWallet = await this.mpcService.createMpcWallet({
      name: `Player Wallet - ${playerId}`,
      thresholdScheme: ThresholdScheme.TSS_2_3,
      participants: [
        { participantId: `player_${playerId}`, ... }, // Device key
        { participantId: `platform_signer`, ... }, // Platform HSM
        { participantId: `recovery_service`, ... }, // Cold storage
      ],
    });
    // Store wallet with default limits
    return this.walletRepository.create({
      playerId,
      mpcWalletId: mpcWallet.walletId,
      dailyWithdrawalLimit: DEFAULT_DAILY_LIMIT, // 10 SOL
      status: PlayerWalletStatus.ACTIVE,
    });
  }
}
```




PlayerWalletService Methods

Method	Purpose	Key Logic
<code>createWallet</code>	Initialize MPC wallet	2-of-3 threshold scheme
<code>deposit</code>	Record external deposit	Verify on-chain signature
<code>withdraw</code>	MPC-signed withdrawal	Rate limit + daily cap
<code>lockFunds</code>	Reserve for escrow	Move to <code>lockedBalance</code>
<code>unlockFunds</code>	Release from escrow	Restore to <code>availableBalance</code>
<code>creditPrize</code>	Award winnings	Direct credit + audit log

Balance Tracking

```
interface WalletBalance {
  availableBalance: string; // Spendable
  lockedBalance: string;    // In active matches
  totalDeposited: string;   // Lifetime in
  totalWithdrawn: string;   // Lifetime out
  totalWinnings: string;    // Prize earnings
}
```

EscrowService

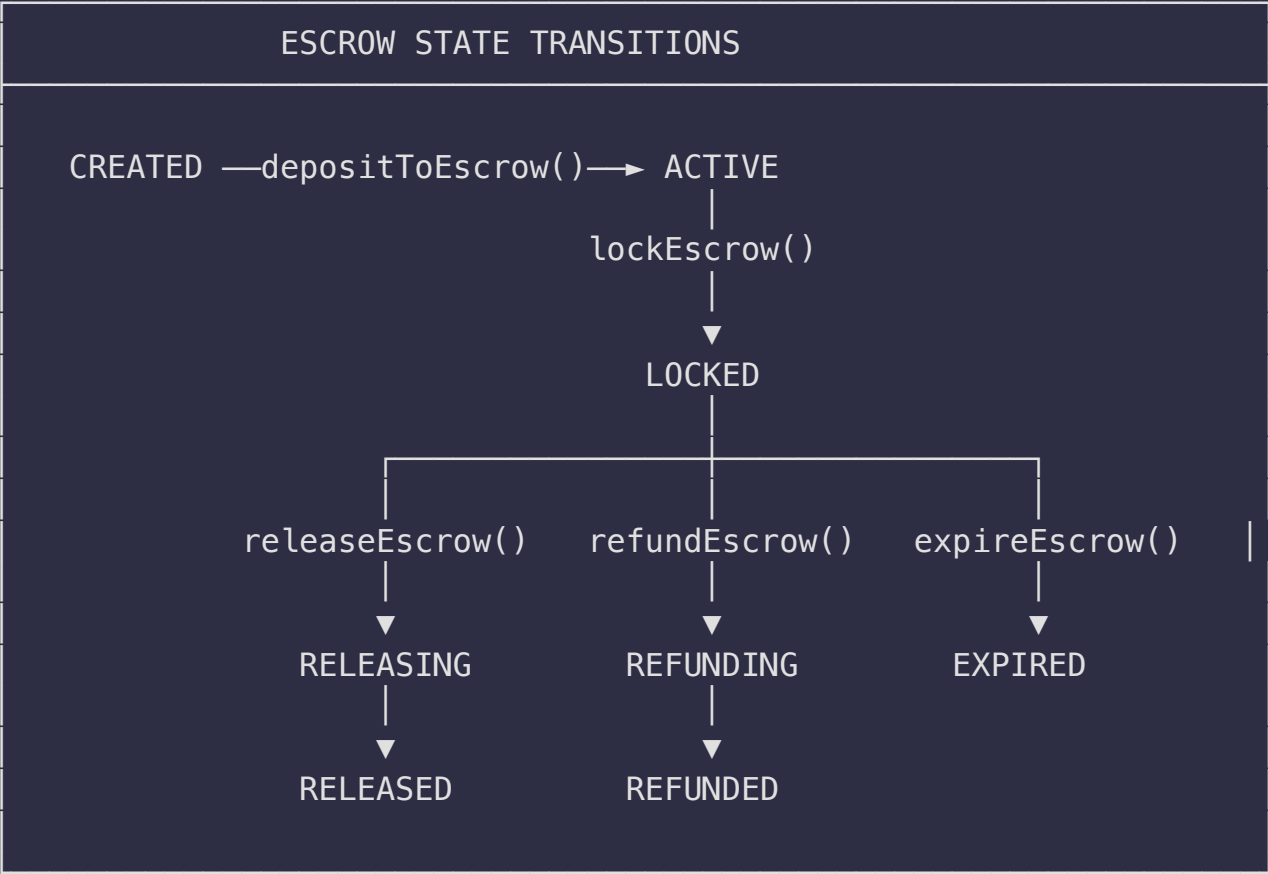
Trustless Fund Management

```
@Injectable()
export class EscrowService {
  // Generate deterministic escrow address (PDA simulation)
  private generateEscrowAddress(sourceType, sourceId): string {
    const seed = `${sourceType}:${sourceId}`;
    return createHash('sha256')
      .update(seed)
      .digest('hex')
      .slice(0, 44);
  }

  async createEscrow(request: CreateEscrowRequest) {
    const escrowAddress = this.generateEscrowAddress(
      request.sourceType,
      request.sourceId
    );
    return this.escrowRepository.create({
      escrowId: `escrow_${randomBytes(8).toString('hex')}`,
      escrowAddress,
      sourceType,           // MATCH or TOURNAMENT
      sourceId,             // match_xxx or tournament_xxx
      platformFeePercent: 5.0,
      status: EscrowStatus.CREATED,
    });
  }
}
```



EscrowService State Machine



MatchmakingService

Core Match Operations

```
@Injectable()
export class MatchmakingService {
  async createMatch(request: CreateMatchRequest): Promise<Match> {
    // 1. Validate entry fee bounds
    const minFee = BigInt(process.env.MIN_ENTRY_FEE || '1000000');
    const maxFee = BigInt(process.env.MAX_ENTRY_FEE || '100000000000');

    // 2. Create escrow account
    const escrow = await this.escrowService.createEscrow({
      sourceType: EscrowSourceType.MATCH,
      sourceId: matchId,
      platformFeePercent,
    });

    // 3. Create match entity
    return this.matchRepository.create({
      matchId,
      gameType,
      entryFee,
      status: MatchStatus.CREATED,
      svmAddress: escrow.escrowAddress,
    });
  }
}
```



MatchmakingService - Join Flow

```
async joinMatch(request: JoinMatchRequest): Promise<MatchParticipant> {
  const match = await this.getMatchById(matchId);

  // 1. Validate match is joinable
  if (!match.isJoinable()) {
    throw new BadRequestException('Match not accepting players');
  }

  // 2. Get player wallet and verify balance
  const wallet = await this.playerWalletService.getWallet(playerId);
  if (BigInt(wallet.availableBalance) < BigInt(match.entryFee)) {
    throw new BadRequestException('Insufficient balance');
  }

  // 3. Lock entry fee in player wallet
  await this.playerWalletService.lockFunds({
    playerId,
    amount: match.entryFee,
    reference: matchId,
  });

  // 4. Deposit to escrow
  await this.escrowService.depositToEscrow({
    escrowId,
    walletId: wallet.id,
    amount: match.entryFee,
  });

  // 5. Update prize pool
  match.prizePool = (BigInt(match.prizePool) + BigInt(entryFee)).toString();
}
```

PrizeDistributionService

Prize Calculation Strategy

```
@Injectable()
export class PrizeDistributionService {
  async distributeMatchPrizes(match, participants): Promise<PrizeDistribution> {
    // 1. Get escrow balance
    const escrowBalance = await this.escrowService.getEscrowBalance(escrowId);
    const totalPrizePool = BigInt(escrowBalance.currentBalance);

    // 2. Calculate platform fee (basis points for precision)
    const platformFee = (totalPrizePool *
      BigInt(Math.round(match.platformFeePercent * 100))) /
      BigInt(10000);

    // 3. Calculate distributable amount
    const distributableAmount = totalPrizePool - platformFee;

    // 4. Calculate individual distributions
    const distributions = this.calculateMatchDistributions(
      match, participants, distributableAmount
    );

    // 5. Release escrow and credit wallets
    await this.escrowService.releaseEscrow({ escrowId, distributions });

    for (const dist of distributions) {
      await this.playerWalletService.creditPrize(
        dist.playerId, dist.amount, match.matchId
      );
    }
  }
}
```

Prize Calculation Methods

Match Distribution Strategies

```
private calculateMatchDistributions(  
  match: Match,  
  participants: MatchParticipant[],  
  distributableAmount: bigint  
): PrizeCalculation[] {  
  const distributions: PrizeCalculation[] = [];  
  
  // Strategy 1: Duel (1v1) – Winner Takes All  
  if (match.gameType === 'duel' && match.result?.winnerIds?.length === 1) {  
    const winner = participants.find(p => p.playerId === winnerId);  
    distributions.push({  
      walletId: winner.walletId,  
      playerId: winner.playerId,  
      placement: 1,  
      amount: distributableAmount.toString(),  
      percentage: 100,  
    });  
  }  
  
  // Strategy 2: Team/Multi-Winner – Equal Split  
  else if (match.result?.winnerIds?.length > 1) {  
    const winnerCount = match.result.winnerIds.length;  
    const prizePerWinner = distributableAmount / BigInt(winnerCount);  
    // ... distribute equally  
  }  
}
```



Tournament Prize Structure

Configurable Payout Tiers

```
private calculateTournamentDistributions(  
  tournament: Tournament,  
  registrations: TournamentRegistration[],  
  distributableAmount: bigint  
) : PrizeCalculation[] {  
  const distributions: PrizeCalculation[] = [];  
  
  // Sort by final placement  
  const ranked = registrations  
    .filter(r => r.finalPlacement != null)  
    .sort((a, b) => a.finalPlacement - b.finalPlacement);  
  
  // Apply prize structure  
  for (const prizeEntry of tournament.prizeStructure) {  
    const registration = ranked.find(  
      r => r.finalPlacement === prizeEntry.place  
    );  
  
    if (registration) {  
      // Support both percentage and fixed amounts  
      const amount = prizeEntry.fixedAmount  
        ? BigInt(prizeEntry.fixedAmount)  
        : (distributableAmount * BigInt(prizeEntry.percentage)) / 100n;  
  
      distributions.push({  
        playerId: registration.playerId,  
        placement: prizeEntry.place,  
        amount: amount.toString(),  
        percentage: prizeEntry.percentage,  
      });  
    }  
  }  
}
```




Fee Calculation Deep Dive

Platform Fee with Basis Points

```
// Why basis points? Precision without floating point errors

// BAD: Floating point multiplication
const fee = totalPool * 0.05; // May have rounding errors

// GOOD: Basis points (100 basis points = 1%)
const platformFee = (totalPrizePool *
  BigInt(Math.round(platformFeePercent * 100))) /
  BigInt(10000);

// Example: 5% fee on 2 SOL (2,000,000,000 lamports)
// platformFeePercent = 5.0
// Math.round(5.0 * 100) = 500 (basis points)
// (2,000,000,000 * 500) / 10000 = 100,000,000 lamports
// = 0.1 SOL platform fee
// Winner receives: 1.9 SOL
```

Pool Size	Fee %	Platform Fee	Winner Prize
2 SOL	5%	0.1 SOL	1.9 SOL

TournamentService

Bracket Generation Algorithm

```
@Injectable()
export class TournamentService {
  async generateBracket(tournamentId: string): Promise<Tournament> {
    const tournament = await this.getTournamentById(tournamentId);
    const registrations = await this.registrationRepository.find({
      where: { tournamentId: tournament.id },
    });

    // Seed players (by skill rating or random)
    const seededPlayers = this.seedPlayers(registrations);

    // Generate bracket based on type
    let bracket: BracketNode[];
    switch (tournament.bracketType) {
      case BracketType.SINGLE_ELIMINATION:
        bracket = this.generateSingleElimination(seededPlayers);
        break;
      case BracketType.DOUBLE_ELIMINATION:
        bracket = this.generateDoubleElimination(seededPlayers);
        break;
      case BracketType.ROUND_ROBIN:
        bracket = this.generateRoundRobin(seededPlayers);
        break;
    }

    // Create match entities for each bracket slot
    await this.createBracketMatches(tournament, bracket);
  }
}
```

Single Elimination Bracket

Seeded Matchups (Standard 1v8, 2v7, etc.)

```
private generateSingleElimination(  
  players: TournamentRegistration[]  
) : BracketMatch[] {  
  const n = players.length;  
  const rounds = Math.ceil(Math.log2(n));  
  const bracket: BracketMatch[] = [];  
  
  // First round matchups (seeded)  
  // Seed 1 vs Seed 8, Seed 2 vs Seed 7, etc.  
  for (let i = 0; i < n / 2; i++) {  
    const highSeed = players[i];  
    const lowSeed = players[n - 1 - i];  
  
    bracket.push({  
      round: 1,  
      position: i,  
      player1Id: highSeed?.playerId,  
      player2Id: lowSeed?.playerId,  
      nextMatchPosition: Math.floor(i / 2),  
    });  
  }  
  
  // Generate placeholder matches for subsequent rounds  
  for (let round = 2; round <= rounds; round++) {  
    const matchesInRound = Math.pow(2, rounds - round);  
    for (let i = 0; i < matchesInRound; i++) {  
      bracket.push({ round, position: i, player1Id: null, player2Id: null });  
    }  
  }  
}
```

MPC Wallet Architecture

Multi-Party Computation (2-of-3 Threshold)





MPC Benefits for Gaming

Feature	Traditional Wallet	MPC Wallet
Single Point of Failure	✗ Yes	✓ No
Key Recovery	✗ Complex	✓ Built-in
Account Takeover Risk	✗ High	✓ Mitigated
User Experience	✗ Seed Phrases	✓ Seamless
Platform Control	✗ None	✓ Fraud Prevention

Key Advantages:

- **No seed phrase exposure** to players
- **Platform co-signing** prevents fraud
- **Recovery service** for lost devices
- **Rate limiting** on withdrawals



Player Wallet Entity

```
@Entity('player_wallets')
export class PlayerWallet {
  @PrimaryGeneratedColumn('uuid')
  id: string;

  @Column({ unique: true })
  playerId: string;

  @Column()
  mpcWalletId: string; // Link to MPC system

  @Column()
  publicKey: string; // Solana address

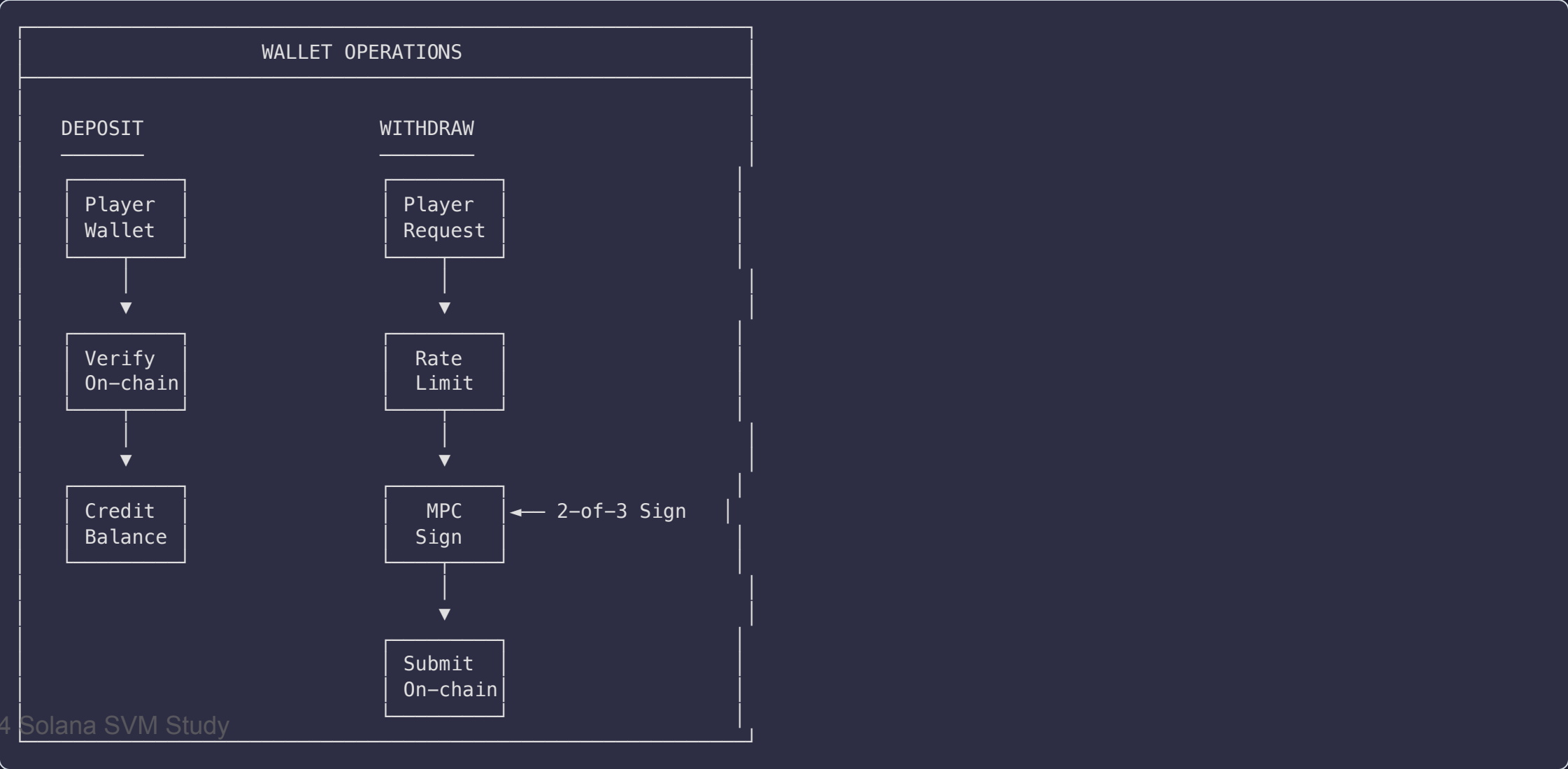
  @Column({ type: 'decimal', precision: 20, scale: 0, default: '0' })
  availableBalance: string; // Lamports

  @Column({ type: 'decimal', precision: 20, scale: 0, default: '0' })
  lockedBalance: string; // In escrow

  @Column({ type: 'decimal', precision: 20, scale: 0, default: '0' })
  dailyWithdrawalLimit: string;

  @Column({ type: 'enum', enum: PlayerWalletStatus })
  status: PlayerWalletStatus;
}
```

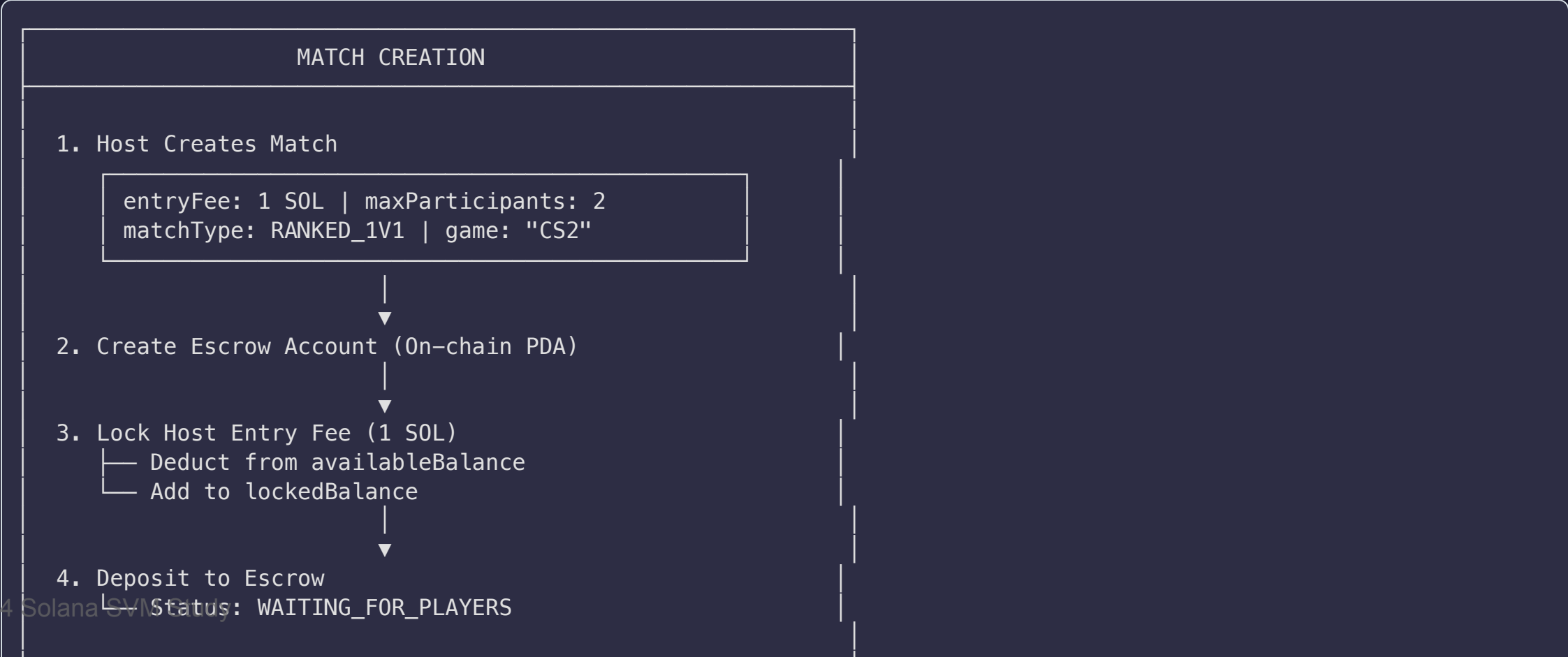
Wallet Operations Flow





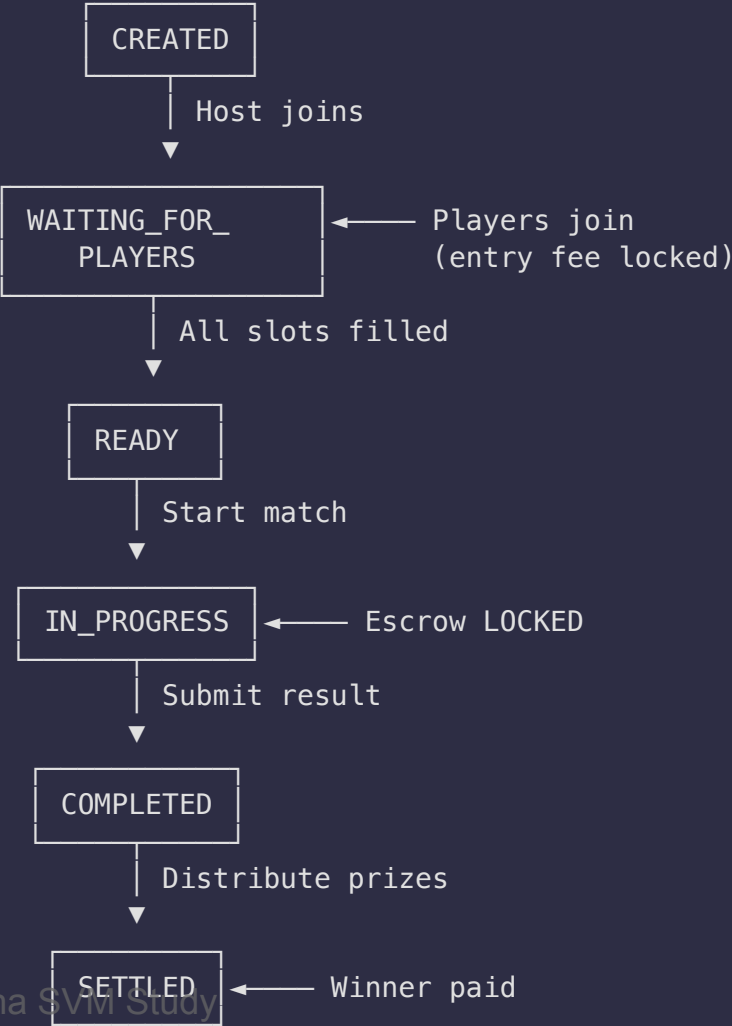
Matchmaking System

Entry Fee & Escrow Flow





Match Lifecycle



Prize Distribution Logic

Winner Takes Pool (minus platform fee)

```
async distributeMatchPrizes(matchId: string) {
  const match = await this.matchRepository.findOne({
    where: { id: matchId },
    relations: ['escrow', 'participants'],
  });

  // Calculate prize pool
  const totalPool = BigInt(match.entryFee) *
    BigInt(match.participants.length);

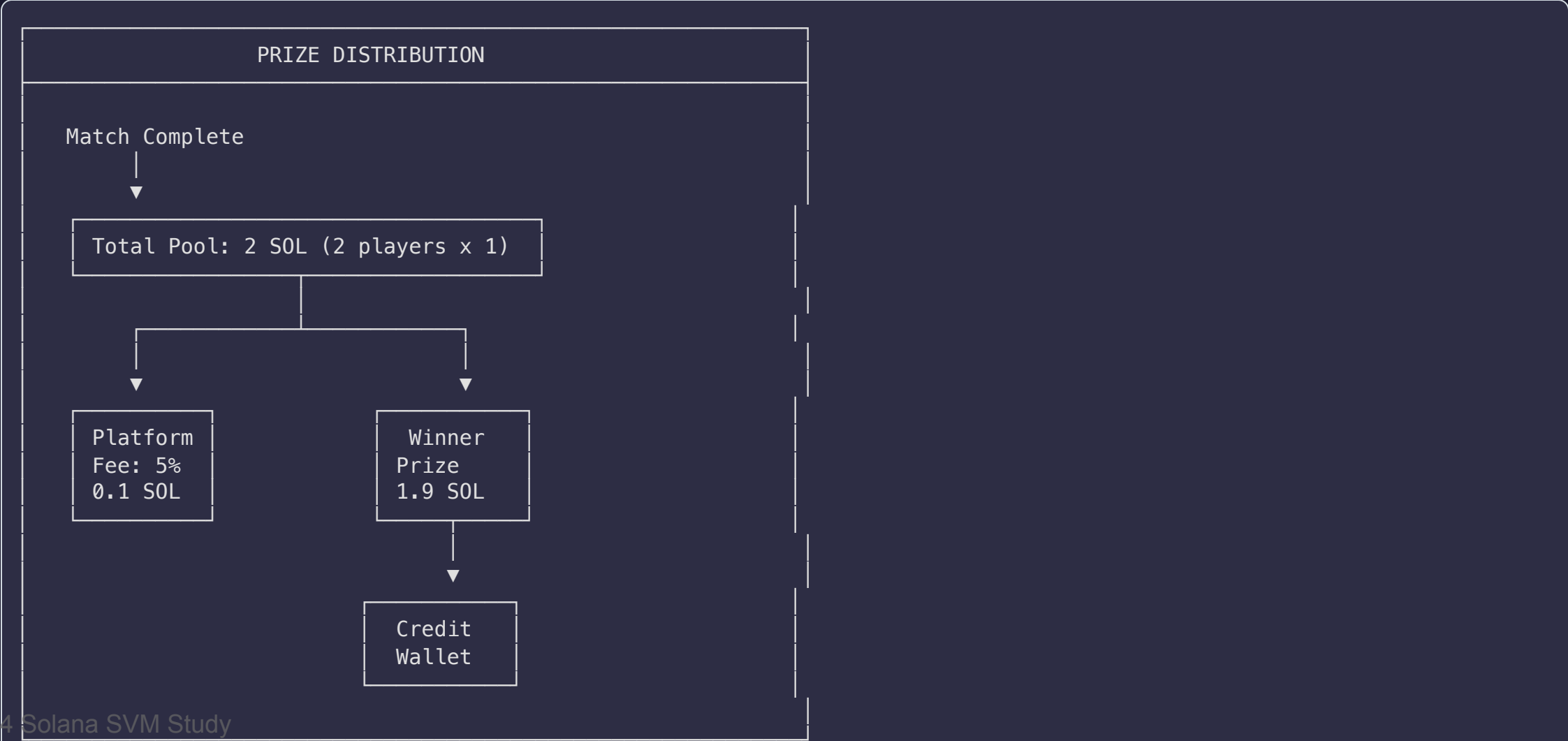
  // Deduct platform fee (e.g., 5%)
  const platformFee = totalPool * BigInt(match.platformFeePercentage) / 100n;
  const winnerPrize = totalPool - platformFee;

  // Credit winner's wallet
  await this.walletService.creditPrize(
    match.winnerId,
    winnerPrize.toString(),
    matchId
  );

  // Release escrow
  await this.escrowRepository.releaseEscrow(matchId);
}
```



Prize Distribution Flow



🔍 Prize Distribution Strategies

Flexible Payout Models for Different Use Cases

PRIZE DISTRIBUTION STRATEGIES		
WINNER TAKES ALL  100% Risk: HIGH Best for: 1v1	TOP 3 SPLIT  60%  30%  10% Risk: MEDIUM Best for: Comp	PERFORMANCE MVP  70%  20%  10% (MVP) Risk: LOW Best for: Casual

Strategy Types Explained

Four Distribution Strategies

Strategy	Description	Risk Level	Best For
WINNER_TAKES_ALL	100% to 1st place	● HIGH	1v1 duels, high-stakes
TOP_3_SPLIT	60/30/10 distribution	● MEDIUM	Competitive tournaments
PERFORMANCE_MVP	70/20/10 with MVP bonus	● LOW	Team games, casual
CUSTOM	User-defined structure	● VARIES	Special events

```
enum PrizeDistributionStrategy {
  WINNER_TAKES_ALL = 'WINNER_TAKES_ALL',
  TOP_3_SPLIT = 'TOP_3_SPLIT',
  PERFORMANCE_MVP = 'PERFORMANCE_MVP',
  CUSTOM = 'CUSTOM',
}

enum PrizeRiskLevel {
  HIGH = 'HIGH',           // Winner takes all
  MEDIUM = 'MEDIUM',      // Top 3 split
  LOW = 'LOW',             // MVP bonus included
}
```

Strategy Configuration

Default Prize Structures by Strategy

```
const PRIZE_STRATEGY_CONFIG: Record<PrizeDistributionStrategy, PrizeStructure[]> = {
  WINNER_TAKES_ALL: [
    { place: 1, percentage: 100, label: 'Winner' }
  ],

  TOP_3_SPLIT: [
    { place: 1, percentage: 60, label: '1st Place' },
    { place: 2, percentage: 30, label: '2nd Place' },
    { place: 3, percentage: 10, label: '3rd Place' },
  ],

  PERFORMANCE_MVP: [
    { place: 1, percentage: 70, label: 'Winner' },
    { place: 2, percentage: 20, label: 'Runner-up' },
    { place: 0, percentage: 10, label: 'MVP Bonus', isMvp: true },
  ],

  CUSTOM: [], // User-defined
};
```



Winner Takes All Strategy

High Risk, High Reward (1v1 Duels)

WINNER TAKES ALL (HIGH RISK)

Prize Pool: 10 SOL
Platform Fee: 5% (0.5 SOL)
Distributable: 9.5 SOL

WINNER

9.5 SOL (100%)

Best for: 1v1 duels, high-stakes matches
Risk: All-or-nothing for participants

Top 3 Split Strategy

Balanced Competition (Tournaments)

TOP 3 SPLIT (MEDIUM RISK)

Prize Pool: 10 SOL | Platform Fee: 5% | Distributable: 9.5

 1ST PLACE: 5.7 SOL (60%)

 2ND PLACE: 2.85 SOL (30%)

 3RD: 0.95 SOL (10%)

Best for: Competitive leagues, standard tournaments

© 2024 Solana SVM Study

48

★ Performance MVP Strategy

Rewarding Excellence (Team Games)

PERFORMANCE MVP (LOW RISK)

Prize Pool: 10 SOL | Distributable: 9.5 SOL

🏆 WINNER: 6.65 SOL (70%)

🥈 RUNNER-UP: 1.9 SOL (20%)

★ MVP: 0.95 SOL (10%)

← Can be ANY participant!

MVP: Best performer regardless of match outcome
Example: Highest K/D, most objectives, best stats

Strategy Implementation

Match Creation with Strategy

```
// Creating a match with specific strategy
const match = await matchmakingService.createMatch({
  gameType: 'battle-royale',
  entryFee: '10000000000', // 1 SOL
  maxParticipants: 10,

  // Strategy configuration
  prizeStrategy: PrizeDistributionStrategy.PERFORMANCE_MVP,
  riskLevel: PrizeRiskLevel.LOW,

  // Optional: Custom structure override
  prizeStructure: [
    { place: 1, percentage: 50, label: 'Champion' },
    { place: 2, percentage: 25, label: 'Runner-up' },
    { place: 3, percentage: 15, label: 'Third' },
    { place: 0, percentage: 10, label: 'MVP', isMvp: true },
  ],
});
```



Submitting Results with MVP

Performance MVP Requires MVP Selection

```
// Submit match result with MVP
await matchmakingService.submitResult(matchId, {
  winnerIds: ['player_1'],
  placements: {
    'player_1': 1,
    'player_2': 2,
    'player_3': 3,
  },

  // Required for PERFORMANCE_MVP strategy
 .mvpPlayerId: 'player_2', // MVP can be non-winner!
 .mvpReason: 'Highest K/D ratio: 15/3',

  metadata: {
    duration: 1800,
    gameVersion: '2.1.0',
  },
});
```



Strategy Comparison Table

Choosing the Right Strategy

Aspect	Winner Takes All	Top 3 Split	Performance MVP
Risk	● High	● Medium	● Low
Players Rewarded	1	3	2-3
MVP Bonus	✗	✗	✓
Best Format	1v1 Duels	Tournaments	Team Games
Skill Ceiling	Very High	High	Moderate
Player Appeal	Competitive	Standard	Casual/Inclusive
Losers Get	Nothing	Top 3 Only	Potential MVP

```
// Query matches by risk level
const casualMatches = await controller.getMatches({
  riskLevel: PrizeRiskLevel.LOW,
  status: MatchStatus.WAITING_FOR_PLAYERS,
});
```



Distribution Calculation

Strategy-Based Prize Calculation

```

calculateDistributionsByStrategy(
  strategy: PrizeDistributionStrategy,
  distributableAmount: bigint,
  participants: Participant[],
  result: MatchResult,
): PrizeCalculation[] {
  const config = PRIZE_STRATEGY_CONFIG[strategy];
  const distributions: PrizeCalculation[] = [];

  for (const prizeEntry of config) {
    if (prizeEntry.isMvp) {
      // MVP bonus - separate from placement
      const mvpPlayer = participants.find(
        p => p.playerId === result.mvpPlayerId
      );
      if (mvpPlayer) {
        distributions.push({
          playerId: mvpPlayer.playerId,
          amount: (distributableAmount * BigInt(prizeEntry.percentage)) / 100n,
          label: 'MVP Bonus',
          isMvp: true,
        });
      }
    } else {
      // Placement-based distribution
      const placedPlayer = participants.find(
        p => result.placements[p.playerId] === prizeEntry.place
      );
      // ... calculate and push
    }
  }
}

```

Risk Level Filtering

Match Discovery by Risk Preference

MATCH DISCOVERY BY RISK LEVEL

Player Profile: "I prefer lower-risk matches"

● LOW RISK MATCHES

- Battle Royale – 10 players – 5 SOL pool
Strategy: PERFORMANCE_MVP
Your worst case: MVP bonus possible
- Team Deathmatch – 8 players – 2 SOL pool
Strategy: TOP_3_SPLIT
Your worst case: 3rd place = 0.19 SOL

● HIGH RISK (Hidden by preference)

Tournament System

Bracket Generation

```
async generateBracket(tournamentId: string): Promise<Tournament> {
  const tournament = await this.tournamentRepository.findOne(/*...*/);
  const registrations = await this.registrationRepository.find({
    where: { tournamentId },
    order: { seed: 'ASC' },
  });

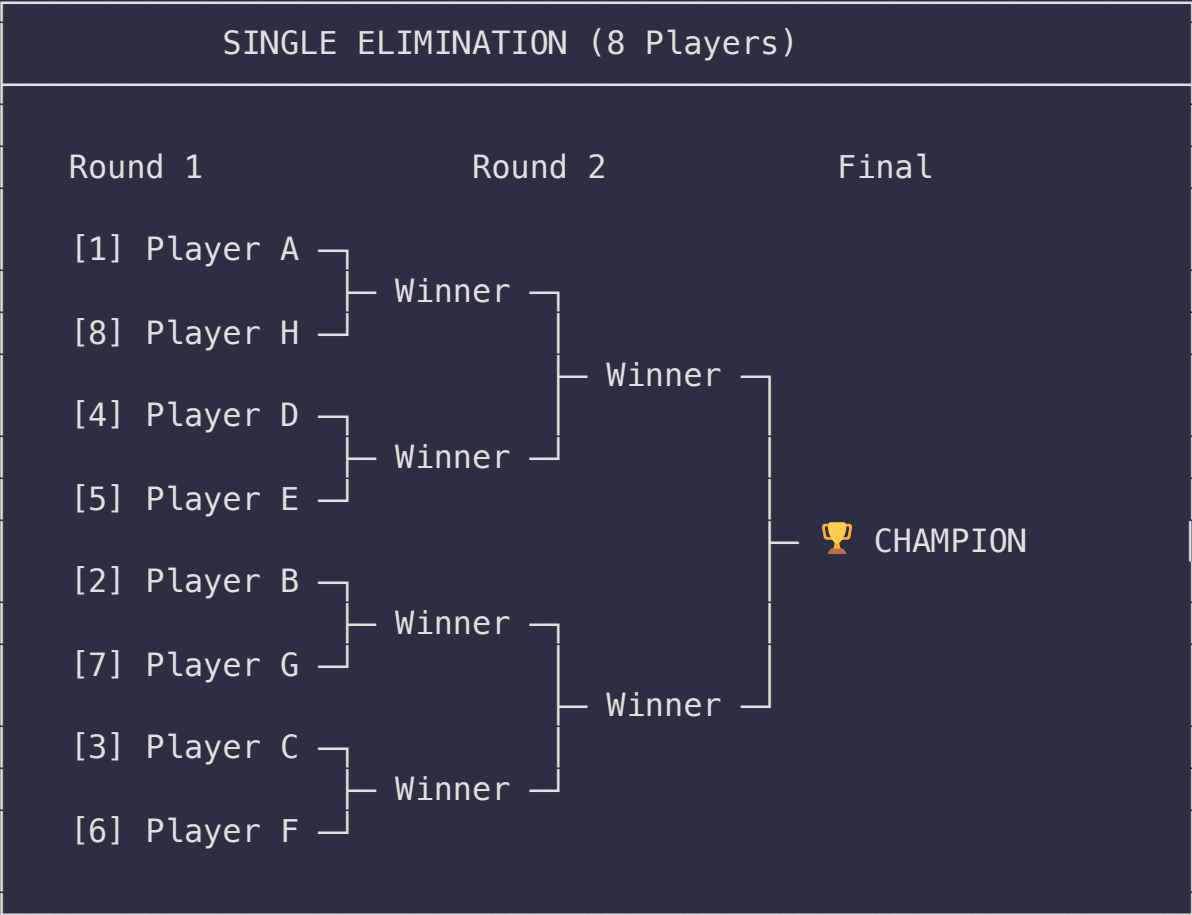
  // Single elimination: seeded matchups
  // 1 vs 8, 2 vs 7, 3 vs 6, 4 vs 5
  const bracket = this.createSingleEliminationBracket(
    registrations,
    tournament.maxParticipants
  );

  // Create match entities for each bracket slot
  const matches = await this.createBracketMatches(
    tournament,
    bracket
  );

  return this.tournamentRepository.save({
    ...tournament,
  });
}
```



Tournament Bracket Structure



Tournament Prize Structure

```
const prizeStructure = {
  1: 50, // 1st place: 50% of pool
  2: 30, // 2nd place: 30% of pool
  3: 20, // 3rd/4th: 20% split
};

// Example: 16 players x 10 SOL entry = 160 SOL pool
// Platform fee (5%): 8 SOL
// Distributable: 152 SOL

// Payouts:
// 1st: 76 SOL
// 2nd: 45.6 SOL
// 3rd: 30.4 SOL
```

Placement	Percentage	Prize (SOL)
 1st	50%	76.0
 2nd	30%	45.6
 3rd	20%	30.4

Escrow Account Entity

```
@Entity('escrow_accounts')
export class EscrowAccount {
  @PrimaryGeneratedColumn('uuid')
  id: string;

  @Column({ type: 'enum', enum: EscrowType })
  type: EscrowType; // MATCH | TOURNAMENT

  @Column()
  reference: string; // match_id or tournament_id

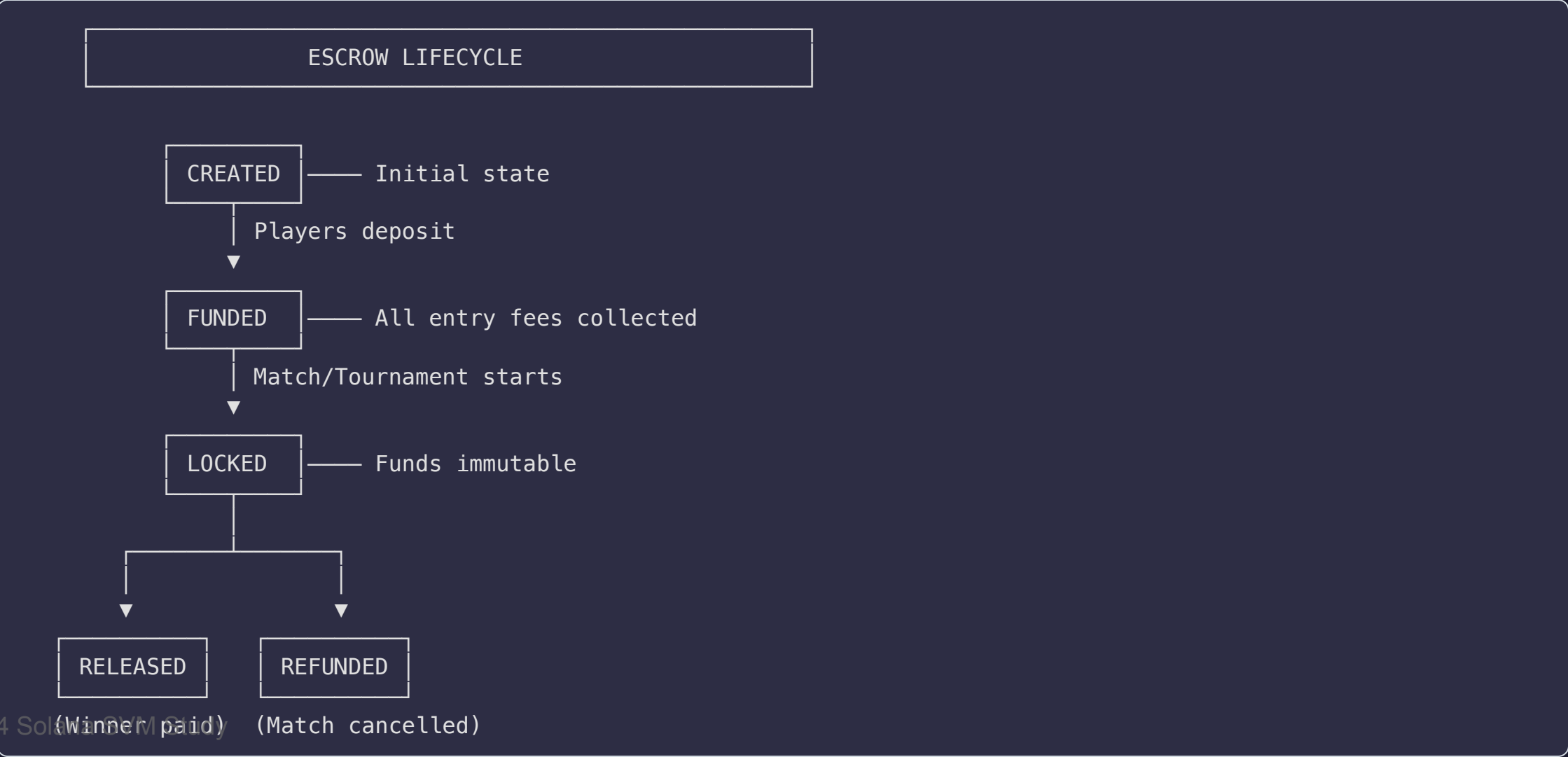
  @Column()
  escrowPda: string; // Program Derived Address

  @Column({ type: 'decimal', precision: 20, scale: 0 })
  amount: string;

  @Column({ type: 'enum', enum: EscrowStatus })
  status: EscrowStatus;
  // CREATED → FUNDED → LOCKED → RELEASED/REFUNDED
}
```



Escrow State Machine



Security Measures

Production Considerations

Wallet Security

-  MPC 2-of-3 threshold signatures
-  Daily withdrawal limits
-  Rate limiting on transactions
-  Account suspension capabilities

Escrow Security

-  On-chain PDA escrow accounts
-  Atomic state transitions
-  Immutable locked funds



Rate Limiting Implementation

```
@Entity('player_wallets')
export class PlayerWallet {
  @Column({ type: 'decimal', precision: 20, scale: 0 })
  dailyWithdrawalLimit: string;

  @Column({ type: 'decimal', precision: 20, scale: 0, default: '0' })
  dailyWithdrawalAmount: string;

  @Column({ type: 'timestamp', nullable: true })
  dailyWithdrawalResetAt: Date;

  canWithdraw(amount: bigint): boolean {
    if (this.status !== PlayerWalletStatus.ACTIVE) return false;

    const available = BigInt(this.availableBalance);
    if (amount > available) return false;

    // Check daily limit
    const dailyUsed = BigInt(this.dailyWithdrawalAmount);
    const dailyLimit = BigInt(this.dailyWithdrawalLimit);

    return (dailyUsed + amount) <= dailyLimit;
  }
}
```



Transaction Types

```
export enum WalletTransactionType {  
  // Inflows  
  DEPOSIT = 'DEPOSIT',           // External deposit  
  PRIZE_WIN = 'PRIZE_WIN',       // Match/tournament win  
  REFUND = 'REFUND',             // Cancelled match refund  
  
  // Outflows  
  WITHDRAWAL = 'WITHDRAWAL',     // External withdrawal  
  ENTRY_FEE = 'ENTRY_FEE',       // Match/tournament entry  
  
  // Internal  
  LOCK = 'LOCK',                 // Lock for escrow  
  UNLOCK = 'UNLOCK',            // Release from escrow  
}
```

Full Audit Trail

Every transaction logged with signatures and timestamps



Complete Match Flow

COMPLETE MATCH FLOW

- 1. CREATE MATCH
 - Host deposits entry fee → Escrow created
- 2. JOIN MATCH
 - Player deposits entry fee → Added to escrow
- 3. START MATCH
 - Escrow LOCKED → Players notified
- 4. PLAY GAME
 - External game server handles gameplay
- 5. SUBMIT RESULT
 - Winner declared → Match COMPLETED
- 6. DISTRIBUTE PRIZES
 - Winner wallet credited → Escrow RELEASED
- 7. SETTLEMENT
 - All balances updated → Match SETTLED



API Endpoints

Matchmaking

Method	Endpoint	Description
POST	<code>/api/matches</code>	Create match
POST	<code>/api/matches/:id/join</code>	Join match
POST	<code>/api/matches/:id/start</code>	Start match
POST	<code>/api/matches/:id/result</code>	Submit result
GET	<code>/api/matches/open</code>	List open matches

Wallets

Method	Endpoint	Description
POST	<code>/api/wallets</code>	Create MPC wallet
GET	<code>/api/wallets/:playerId/balance</code>	Get balance

API Endpoints (cont.)

Tournaments

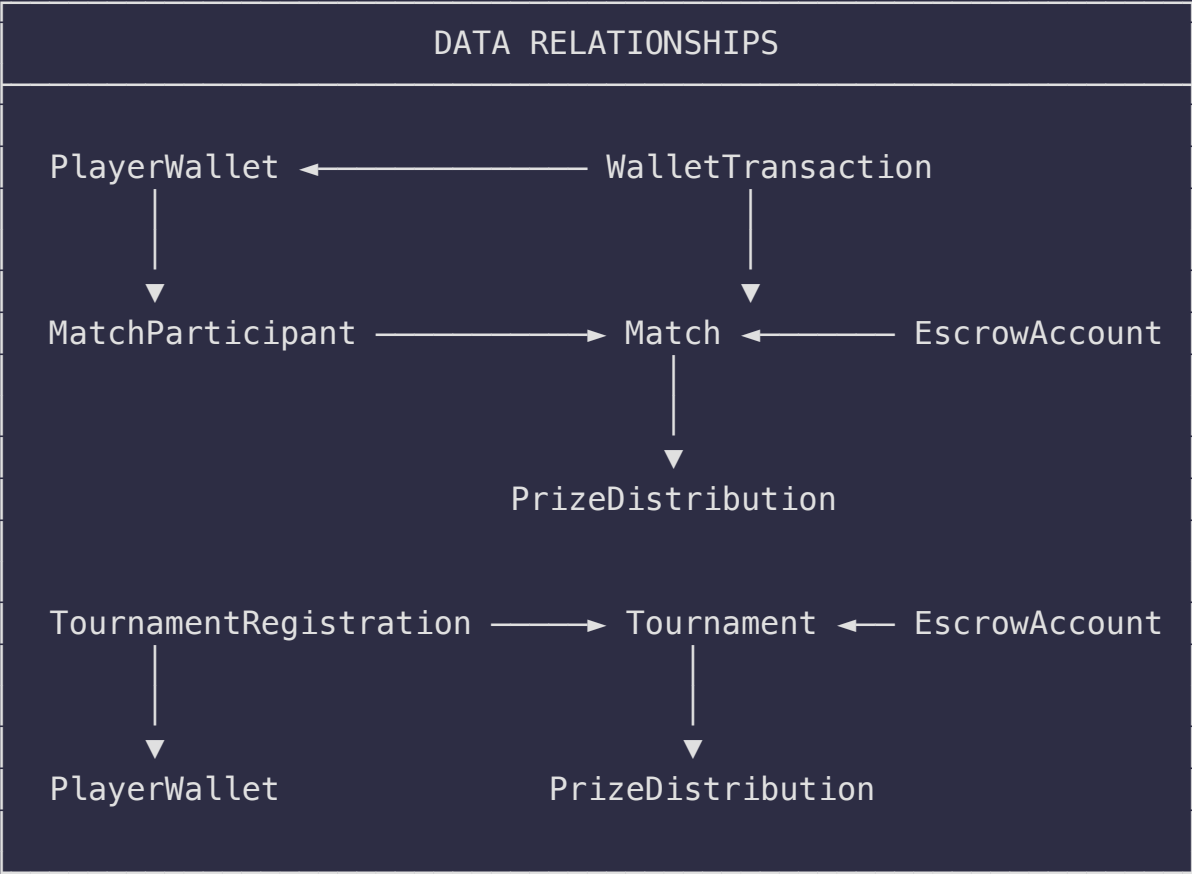
Method	Endpoint	Description
POST	/api/tournaments	Create tournament
POST	/api/tournaments/:id/register	Register player
POST	/api/tournaments/:id/bracket	Generate bracket
POST	/api/tournaments/:id/start	Start tournament
GET	/api/tournaments/:id/leaderboard	Get standings

Prizes

Method	Endpoint	Description
POST	/api/prizes/match/:id/distribute	Pay match prizes
POST	/api/prizes/tournament/:id/distribute	Pay tournament



Data Model Overview





Testing Strategy

Unit Tests

```
describe('EscrowService', () => {
  it('should create escrow for match', async () => {
    const result = await escrowService.createEscrow({
      type: EscrowType.MATCH,
      reference: 'match_123',
      tokenMint: SOL_MINT,
    });

    expect(result.status).toBe(EscrowStatus.CREATED);
    expect(result.escrowPda).toBeDefined();
  });

  it('should reject release for unlocked escrow', async () => {
    await expect(
      escrowService.releaseEscrow('escrow_unlocked')
    ).rejects.toThrow(BadRequestException);
  });
});
```



Integration Testing

```
describe('Match E2E', () => {
  it('should complete full match flow', async () => {
    // 1. Create wallets for both players
    const wallet1 = await walletService.createWallet({ playerId: 'p1' });
    const wallet2 = await walletService.createWallet({ playerId: 'p2' });

    // 2. Deposit funds
    await walletService.deposit({ playerId: 'p1', amount: '20000000000' });
    await walletService.deposit({ playerId: 'p2', amount: '20000000000' });

    // 3. Create and join match
    const match = await matchService.createMatch({
      hostPlayerId: 'p1',
      entryFee: '10000000000',
    });
    await matchService.joinMatch(match.id, 'p2');

    // 4. Start and complete
    await matchService.startMatch(match.id);
    await matchService.submitResult(match.id, { winnerId: 'p1' });

    // 5. Distribute prizes
    await prizeService.distributeMatchPrizes(match.id);

    // 6. Verify balances
    const balance1 = await walletService.getBalance('p1');
    expect(balance1.availableBalance).toBe('29000000000'); // Won
  });
});
```



Monitoring & Observability

Key Metrics to Track

```
// Custom metrics for prize distribution
@Injectable()
export class PrizeDistributionService {
  private readonly prizeDistributedCounter = new Counter({
    name: 'esports_prizes_distributed_total',
    help: 'Total prizes distributed',
    labelNames: ['type', 'status'],
  });

  private readonly prizeAmountGauge = new Gauge({
    name: 'esports_prize_amount_sol',
    help: 'Prize amount in SOL',
    labelNames: ['type'],
  });

  async distributePrize(/*...*/) {
    // ... distribution logic

    this.prizeDistributedCounter.inc({ type: 'match', status: 'success' });
    this.prizeAmountGauge.set({ type: 'match' }, prizeInSol);
  }
}
```



Key Metrics Dashboard

Metric	Description	Alert Threshold
<code>escrow_balance_total</code>	Total SOL in escrow	> 1000 SOL
<code>match_completion_rate</code>	% matches completed	< 90%
<code>prize_distribution_latency</code>	Time to distribute	> 30s
<code>mpc_signing_failures</code>	Failed MPC signs	> 0
<code>withdrawal_rate_limit_hits</code>	Rate limit triggers	> 100/hr

Alerts

- 🚨 Escrow balance mismatch
- 🚨 MPC signing failure rate > 1%
- 🚨 Prize distribution backlog
- 🚨 Suspicious withdrawal patterns

Best Practices Summary

1. Security First

- Always use MPC for player wallets
- Lock escrow before match starts
- Implement withdrawal limits

2. Atomic Operations

- Use database transactions
- Handle partial failures gracefully
- Implement idempotency keys

3. Auditability

- Log all financial transactions

Future Enhancements

Roadmap Items

1. Cross-Game Wallet

- Single wallet across multiple games
- Unified balance management

2. NFT Integration

- Tournament trophy NFTs
- Achievement badges

3. DAO Governance

- Community-driven prize pools
- Voting on platform fees



Advanced Calculation Patterns

BigInt Safety for Financial Operations

```
// CRITICAL: Always use BigInt for lamport calculations
// JavaScript Number.MAX_SAFE_INTEGER = 9,007,199,254,740,991
// 1 SOL = 1,000,000,000 lamports
// Max safe: ~9,007 SOL with regular Number

// BAD: Number overflow risk
const pool = 10000000000 * 2; // Potential precision loss

// GOOD: BigInt for all financial math
const pool = BigInt('10000000000') * BigInt(2);

// Division truncates (floor), handle remainder
const prizePerWinner = distributableAmount / BigInt(winnerCount);
const remainder = distributableAmount % BigInt(winnerCount);
// Option: Add remainder to 1st place winner
```



Atomic Transaction Pattern

Database + Blockchain Consistency

```
async distributeMatchPrizes(match: Match): Promise<PrizeDistribution> {
  const queryRunner = this.dataSource.createQueryRunner();
  await queryRunner.connect();
  await queryRunner.startTransaction();

  try {
    // 1. Create prize distribution record (PROCESSING)
    const prizeDistribution = await queryRunner.manager.save(
      PrizeDistribution, { status: 'PROCESSING', ... }
    );

    // 2. Release escrow (on-chain operation)
    await this.escrowService.releaseEscrow({ escrowId, distributions });

    // 3. Credit each winner's wallet
    for (const dist of distributions) {
      await this.playerWalletService.creditPrize(dist.playerId, dist.amount);

      // 4. Update distribution status individually
      dist.status = 'COMPLETED';
      dist.processedAt = new Date();
    }

    // 5. Commit all database changes
    prizeDistribution.status = 'COMPLETED';
    await queryRunner.manager.save(prizeDistribution);
    await queryRunner.commitTransaction();

    return prizeDistribution;
  } catch (error) {
    await queryRunner.rollbackTransaction();
    throw error;
  }
}
```

Error Recovery Strategies

Partial Distribution Handling

```
// Prize distribution supports partial success
for (const dist of distributions) {
  try {
    await this.playerWalletService.creditPrize(
      dist.playerId, dist.amount, sourceId
    );

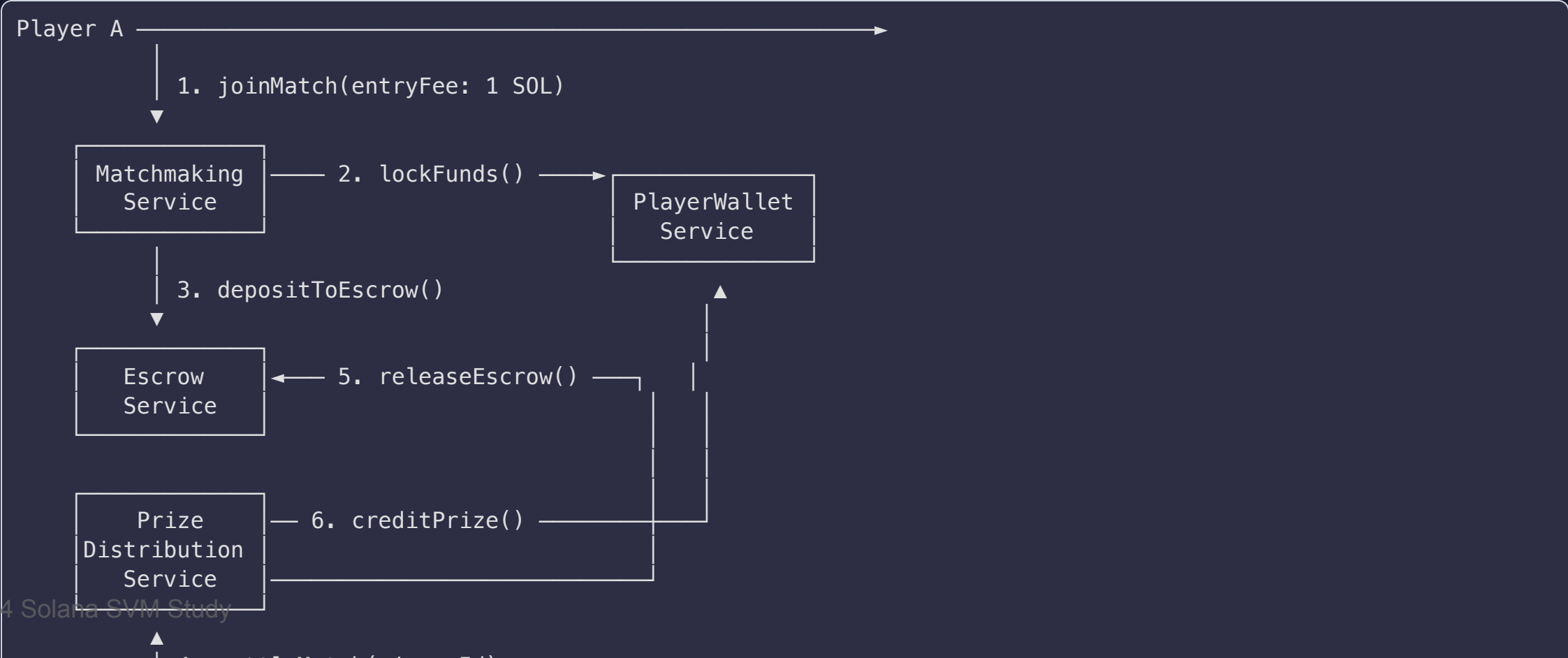
    distEntry.status = 'completed';
    distEntry.processedAt = new Date();
    totalDistributed += BigInt(dist.amount);

  } catch (error) {
    // Log but continue to next recipient
    this.logger.error(`Failed to credit ${dist.playerId}`, error);

    distEntry.status = 'failed';
    distEntry.failureReason = error.message;
    // Can be retried later via admin action
  }
}
```

Service Interaction Diagram

Complete Prize Distribution Flow



❖ Business Rules: Entry Fee Validation

Configurable Fee Boundaries

```
// Environment-based fee limits (lamports)
const MIN_ENTRY_FEE = BigInt(process.env.MIN_ENTRY_FEE || '1000000'); // 0.001 SOL
const MAX_ENTRY_FEE = BigInt(process.env.MAX_ENTRY_FEE || '100000000000'); // 100 SOL

// Validation rules applied at match creation
async validateEntryFee(entryFee: string, gameType: string): Promise<void> {
  const fee = BigInt(entryFee);

  // Rule 1: Minimum threshold (prevent spam)
  if (fee < MIN_ENTRY_FEE) {
    throw new BadRequestException(`Entry fee below minimum: ${MIN_ENTRY_FEE}`);
  }

  // Rule 2: Maximum cap (risk management)
  if (fee > MAX_ENTRY_FEE) {
    throw new BadRequestException(`Entry fee exceeds maximum: ${MAX_ENTRY_FEE}`);
  }

  // Rule 3: Game-specific limits
  const gameLimit = await this.getGameFeeLimit(gameType);
  if (fee > gameLimit) {
    throw new BadRequestException(`Fee exceeds game limit for ${gameType}`);
  }
}
```

Business Rules: Player Eligibility

Multi-Factor Verification

```
async verifyPlayerEligibility(playerId: string, matchId: string): Promise<void> {
  const player = await this.playerService.getPlayer(playerId);
  const wallet = await this.walletService.getWallet(playerId);
  const match = await this.matchService.getMatch(matchId);

  // Rule 1: Account status
  if (player.status !== PlayerStatus.ACTIVE) {
    throw new ForbiddenException('Account suspended or inactive');
  }

  // Rule 2: KYC verification (for high-value matches)
  if (BigInt(match.entryFee) > KYC_THRESHOLD && !player.kycVerified) {
    throw new ForbiddenException('KYC required for high-value matches');
  }

  // Rule 3: Region restrictions
  if (match.allowedRegions && !match.allowedRegions.includes(player.region)) {
    throw new ForbiddenException('Match restricted to specific regions');
  }

  // Rule 4: Skill rating bounds (prevent smurfing)
  if (match.minRating && player.skillRating < match.minRating) {
    throw new ForbiddenException(`Minimum rating required: ${match.minRating}`);
  }

  // Rule 5: Concurrent match limit
  const activeMatches = await this.matchService.getActiveMatches(playerId);
```

Business Rules: Eligibility Criteria

Player Status Requirements

Criterion	Requirement	Enforcement
Account Age	> 24 hours	Prevent bot spam
Email Verified	Required	Identity verification
Phone Verified	High-stakes only	2FA for withdrawals
KYC Level 1	Matches > 10 SOL	Regulatory compliance
KYC Level 2	Matches > 50 SOL	Enhanced due diligence
Skill Rating	Within ± 200 ELO	Fair matchmaking
Active Bans	None	Anti-cheat compliance
Pending Disputes	None	Dispute resolution first

```
© 2024 Solana SVM Study
// Eligibility flags stored on player entity
interface PlayerEligibility {
  canPlayCasual: boolean; // Basic account status
```

Business Rules: Refund Policies

Scenario-Based Refund Logic

```
enum RefundReason {  
  MATCH_CANCELLED = 'MATCH_CANCELLED',           // Full refund  
  OPPONENT_NO_SHOW = 'OPPONENT_NO_SHOW',          // Full refund + bonus  
  TECHNICAL_FAILURE = 'TECHNICAL_FAILURE',        // Full refund  
  PLAYER_DISCONNECT = 'PLAYER_DISCONNECT',         // Partial (time-based)  
  DISPUTE_RESOLVED = 'DISPUTE_RESOLVED',          // Per arbitration  
  TOURNAMENT_CANCELLED = 'TOURNAMENT_CANCELLED',   // Full refund  
}  
  
async processRefund(escrowId: string, reason: RefundReason): Promise<void> {  
  const escrow = await this.escrowService.getEscrow(escrowId);  
  
  switch (reason) {  
    case RefundReason.MATCH_CANCELLED:  
      // Rule: 100% refund to all participants  
      await this.refundAllParticipants(escrow, 100);  
      break;  
  
    case RefundReason.OPPONENT_NO_SHOW:  
      // Rule: 100% refund + 10% bonus from platform pool  
      await this.refundWithBonus(escrow, waitingPlayer, 10);  
      break;  
  
    case RefundReason.PLAYER_DISCONNECT:  
      // Rule: Partial refund based on match progress  
      const refundPercent = this.calculateDisconnectRefund(escrow);  
      await this.refundWithBonus(escrow, waitingPlayer, refundPercent);  
      break;  
  }  
}
```




Business Rules: Refund Matrix

Refund Percentage by Scenario

Scenario	Refund %	Waiting Time	Platform Action
Match cancelled (< 1 min)	100%	Immediate	Auto-refund
Match cancelled (> 1 min)	100%	5 min	Admin approval
Opponent no-show	100% + bonus	10 min wait	Auto after timeout
Technical failure	100%	24 hr review	Manual verification
Disconnect (< 25% match)	75%	48 hr	Dispute period
Disconnect (25-50% match)	50%	48 hr	Dispute period
Disconnect (> 50% match)	0%	N/A	Forfeit
Rage quit	0%	N/A	Forfeit + penalty
Tournament cancelled	100%	24 hr	Batch refund

Business Rules: Match Cancellation

Cancellation Conditions & Consequences

```
enum CancellationReason {
  HOST_CANCELLED = 'HOST_CANCELLED',
  INSUFFICIENT_PLAYERS = 'INSUFFICIENT_PLAYERS',
  MATCH_TIMEOUT = 'MATCH_TIMEOUT',
  ADMIN_CANCELLED = 'ADMIN_CANCELLED',
  SERVER_UNAVAILABLE = 'SERVER_UNAVAILABLE',
}

async cancelMatch(matchId: string, reason: CancellationReason): Promise<void> {
  const match = await this.matchService.getMatch(matchId);

  // Rule 1: Only cancel if not IN_PROGRESS or later
  if (!match.isCancellable()) {
    throw new BadRequestException('Match cannot be cancelled after start');
  }

  // Rule 2: Host penalty for frequent cancellations
  if (reason === CancellationReason.HOST_CANCELLED) {
    const recentCancels = await this.getCancellationCount(match.hostId, 24);
    if (recentCancels >= 3) {
      await this.applyHostPenalty(match.hostId); // 24hr match creation ban
    }
  }

  // Rule 3: Auto-refund all participants
  await this.escrowService.refundEscrow({
    escrowId: match.escrowId,
    reason,
    distributions: match.participants.map(p => ({
      playerId: p.playerId,
      amount: match.entryFee,
    })),
  });
}
```



Business Rules: Cancellation Penalties

Host Accountability System

Cancellations (24hr)	Penalty	Duration
1-2	Warning	N/A
3-4	24hr match creation ban	24 hours
5+	7-day ban + rating penalty	7 days
Repeat offender	Account review	Indefinite

```
// Penalty escalation logic
async applyHostPenalty(playerId: string): Promise<void> {
  const cancelHistory = await this.getCancellationHistory(playerId);
  const recent = cancelHistory.filter(c => c.age < 7 * 24 * 60 * 60 * 1000);

  if (recent.length >= 10) {
    // Severe: Account review required
    await this.flagForReview(playerId, 'EXCESSIVE_CANCELLATIONS');
    await this.suspendMatchCreation(playerId, 'INDEFINITE');
  } else if (recent.length >= 5) {
    // Major: 7-day ban + rating penalty
  }
```

Business Rules: Dispute Resolution

Multi-Tier Dispute System

```
enum DisputeStatus {
    OPENED = 'OPENED',
    EVIDENCE_COLLECTION = 'EVIDENCE_COLLECTION',
    UNDER_REVIEW = 'UNDER_REVIEW',
    ARBITRATION = 'ARBITRATION',
    RESOLVED = 'RESOLVED',
    APPEALED = 'APPEALED',
}

interface Dispute {
    disputeId: string;
    matchId: string;
    claimantId: string;    // Player filing dispute
    respondentId: string;  // Other player
    category: DisputeCategory;
    evidence: Evidence[];
    status: DisputeStatus;
    tier: DisputeTier;      // AUTO → SUPPORT → ARBITRATION
    resolution?: DisputeResolution;
}

enum DisputeCategory {
    CHEATING = 'CHEATING',
    DISCONNECT = 'DISCONNECT',
    WRONG_RESULT = 'WRONG_RESULT',
}
```

Business Rules: Dispute Tiers

Escalation Ladder



Required Evidence Types

Business Rules: Timeout Handling

Match Lifecycle Timeouts

```
// Configurable timeout values
const TIMEOUTS = {
  MATCH_FILL: 30 * 60 * 1000,      // 30 min to fill match
  PLAYER_READY: 5 * 60 * 1000,     // 5 min ready check
  MATCH_START: 10 * 60 * 1000,     // 10 min to connect to game
  RESULT_SUBMIT: 30 * 60 * 1000,   // 30 min after match end
  DISPUTE_WINDOW: 24 * 60 * 60 * 1000, // 24 hr dispute window
  PRIZE_CLAIM: 30 * 24 * 60 * 60 * 1000, // 30 days to claim
};

@Cron('*/*5 * * * *') // Every 5 minutes
async handleMatchTimeouts(): Promise<void> {
  // Rule 1: Cancel unfilled matches
  const unfilledMatches = await this.matchRepository.find({
    where: {
      status: MatchStatus.WAITING_FOR_PLAYERS,
      createdAt: LessThan(new Date(Date.now() - TIMEOUTS.MATCH_FILL)),
    },
  });
};
```

```
function cancelMatch of unfilledMatches) {
  await this.cancelMatch(match.id, CancellationReason.MATCH_TIMEOUT);
}
```

Business Rules: Timeout Actions

Automatic Timeout Handlers

Timeout	Trigger	Action	Notification
Match Fill (30m)	No players join	Cancel + refund	Host notified
Ready Check (5m)	Player not ready	Kick + refund	Warning sent
Game Connect (10m)	No connection	Auto-forfeit	Opponent wins
Result Submit (30m)	No result	Admin escalation	Both notified
Dispute Window (24h)	No dispute	Distribute prizes	Final settlement
Prize Claim (30d)	Unclaimed prize	Move to reserve	Last chance email

```
// Ready check timeout handling
async handleReadyTimeout(matchId: string): Promise<void> {
  const match = await this.matchService.getMatch(matchId);
  const notReady = match.participants.filter(p => !p.isReady);
  for (const player of notReady) {
    // Remove player and refund
  }
}
```


Business Rules: Tournament Registration

Registration Constraints

```
async registerForTournament(  
  tournamentId: string,  
  playerId: string  
) : Promise<TournamentRegistration> {  
  const tournament = await this.getTournament(tournamentId);  
  const player = await this.playerService.getPlayer(playerId);  
  
  // Rule 1: Registration window  
  if (new Date() > tournament.registrationDeadline) {  
    throw new BadRequestException('Registration closed');  
  }  
  
  // Rule 2: Capacity check  
  const registrations = await this.getRegistrationCount(tournamentId);  
  if (registrations >= tournament.maxParticipants) {  
    // Add to waitlist if enabled  
    if (tournament.waitlistEnabled) {  
      return this.addToWaitlist(tournamentId, playerId);  
    }  
    throw new BadRequestException('Tournament full');  
  }  
  
  // Rule 3: Eligibility (skill bounds, KYC, etc.)  
  await this.verifyTournamentEligibility(player, tournament);  
  
  // Rule 4: No duplicate registration  
  const existing = await this.findRegistration(tournamentId, playerId);  
  if (existing) {  
    throw new ConflictException('Already registered');  
  }  
}
```



Business Rules: Tournament Constraints

Registration Requirements

Constraint	Rule	Exception
Registration Window	Opens 7d before, closes 1hr before	Admin override
Min Players	Tournament starts only if min reached	Cancel + refund
Max Players	First-come-first-served	Waitlist fills drops
Entry Fee Lock	Locked at registration	Refund if withdrawn early
Withdrawal Deadline	24hr before start	100% refund
Late Withdrawal	< 24hr before start	50% refund
No-Show	Not present at start	0% refund + ban

```
// Withdrawal refund calculation
calculateWithdrawalRefund(tournament: Tournament, requestTime: Date): number {
  const hoursUntilStart = (tournament.startTime - requestTime) / (60 * 60 * 1000);

  if (hoursUntilStart > 24) return 100; // Full refund
```

Business Rules: Prize Pool Guarantees

Guaranteed Prize Pools (GPP)

```
interface GuaranteedPrizePool {
  guaranteedAmount: string; // Minimum prize pool
  actualPool: string; // Entry fees collected
  overlayAmount: string; // Platform covers shortfall
  isGuaranteed: boolean;
}

async calculatePrizePool(tournament: Tournament): Promise<GuaranteedPrizePool> {
  const registrations = await this.getRegistrationCount(tournament.id);
  const actualPool = BigInt(tournament.entryFee) * BigInt(registrations);
  const guaranteedAmount = BigInt(tournament.guaranteedPrizePool || '0');

  // Platform covers shortfall (overlay)
  const overlayAmount = guaranteedAmount > actualPool
    ? guaranteedAmount - actualPool
    : BigInt(0);

  return {
    guaranteedAmount: guaranteedAmount.toString(),
    actualPool: actualPool.toString(),
    overlayAmount: overlayAmount.toString(),
    isGuaranteed: tournament.guaranteedPrizePool != null,
  };
}
```

Business Rules: GPP Economics

Guaranteed Pool Scenarios

Scenario	Entry Fee	Expected	Actual	Overlay	Result
Under-filled	10 SOL	100 players	80	200 SOL	GPP met
Exactly met	10 SOL	100 players	100	0 SOL	GPP met
Over-filled	10 SOL	100 players	120	0 SOL	+200 SOL

```
// Prize pool calculation with GPP
async finalizePrizePool(tournament: Tournament): Promise<void> {
  const { actualPool, guaranteedAmount, overlayAmount } =
    await this.calculatePrizePool(tournament);

  // If overlay required, transfer from platform reserve
  if (BigInt(overlayAmount) > 0) {
    await this.transferFromPlatformReserve(
      tournament.escrowId,
      overlayAmount
    );

    // Log overlay for accounting
    await this.logOverlay({
```

Business Rules: Anti-Collusion

Collusion Detection Mechanisms

```
interface CollusionIndicator {
  type: CollusionType;
  severity: 'LOW' | 'MEDIUM' | 'HIGH';
  confidence: number; // 0-100
  evidence: string[];
}

enum CollusionType {
  CHIP_DUMPING = 'CHIP_DUMPING', // Intentionally losing
  WIN_TRADING = 'WIN_TRADING', // Alternating wins
  MULTI_ACCOUNTING = 'MULTI_ACCOUNTING', // Same person, multiple accounts
  SOFT_PLAYING = 'SOFT_PLAYING', // Not competing when matched
  GHOSTING = 'GHOSTING', // External assistance
}

async detectCollusion(matchId: string): Promise<CollusionIndicator[]> {
  const indicators: CollusionIndicator[] = [];
  const match = await this.matchService.getMatch(matchId);

  // Check 1: Win trading (same players, alternating wins)
  const history = await this.getMatchHistory(match.participants);
  if (this.detectWinTrading(history)) {
    indicators.push({
      type: CollusionType.WIN_TRADING,
      severity: 'HIGH',
      confidence: 85,
      evidence: ['10 matches, 5-5 split, suspicious timing'],
    });
  }
}
```

Business Rules: Collusion Prevention

Detection Rules & Actions

Indicator	Detection Method	Action	Appeal
Same IP	Network fingerprint	Block match	Proof of separate identity
Device match	Hardware ID	Account link warning	Explain shared device
Win trading	Win/loss pattern	Flag for review	Match history audit
Chip dumping	Game log analysis	Suspend + investigate	Replay review
Soft playing	Performance metrics	Warning	Prove competitive play

```
// Pre-match collusion check
async validateMatchParticipants(matchId: string): Promise<void> {
  const match = await this.matchService.getMatch(matchId);
  const participants = match.participants;

  for (let i = 0; i < participants.length; i++) {
    for (let j = i + 1; j < participants.length; j++) {
      const p1 = participants[i];
      const p2 = participants[j];
```

Business Rules: Abandonment Handling

Match Abandonment Rules

```
enum AbandonmentType {
  DISCONNECT = 'DISCONNECT',    // Network issues
  QUIT = 'QUIT',                 // Voluntary exit
  AFK = 'AFK',                   // Away from keyboard
  CRASH = 'CRASH',               // Game/system crash
}

async handleAbandonment(
  matchId: string,
  playerId: string,
  type: AbandonmentType
): Promise<void> {
  const match = await this.matchService.getMatch(matchId);

  // Rule 1: Grace period for reconnection
  if (type === AbandonmentType.DISCONNECT || type === AbandonmentType.CRASH) {
    const gracePeriod = match.reconnectGracePeriod || 3 * 60 * 1000; // 3 min

    await this.startReconnectTimer(matchId, playerId, gracePeriod);
    return; // Wait for reconnect before forfeit
  }

  // Rule 2: Immediate forfeit for quit/AFK
  if (type === AbandonmentType.QUIT || type === AbandonmentType.AFK) {
    await this.forfeitPlayer(matchId, playerId);
    await this.applyAbandonmentPenalty(playerId, type);
  }
}
```

Business Rules: Abandonment Penalties

Progressive Penalty System

Abandonments (7d)	Type	Penalty	Cooldown
1st	Disconnect	Warning	None
1st	Quit	-10 rating	None
2nd	Any	-25 rating	1hr queue ban
3rd	Any	-50 rating	6hr queue ban
4th	Any	-100 rating	24hr queue ban
5+	Any	Account review	7d ban

```
async applyAbandonmentPenalty(  
  playerId: string,  
  type: AbandonmentType  
): Promise<void> {  
  const recentAbandons = await this.getAbandonments(playerId, 7 * 24 * 60 * 60);  
  const count = recentAbandons.length + 1;
```




Business Rules: Platform Fee Structure

Dynamic Fee Configuration

```
interface FeeStructure {
  baseFeePercent: number; // Default platform fee
  volumeDiscounts: VolumeDiscount[];
  promotionalRates: PromotionalRate[];
  gameSpecificFees: Map<string, number>;
}

// Volume-based discounts for high-value tournaments
const VOLUME_DISCOUNTS: VolumeDiscount[] = [
  { minPool: '0', maxPool: '10000000000', feePercent: 5.0 }, // < 100 SOL: 5%
  { minPool: '10000000000', maxPool: '100000000000', feePercent: 4.0 }, // 100-1000: 4%
  { minPool: '100000000000', maxPool: null, feePercent: 3.0 }, // > 1000 SOL: 3%
];

async calculatePlatformFee(
  prizePool: bigint,
  gameType: string,
  eventType: string
): Promise<bigint> {
  // Base fee from volume tier
  let feePercent = this.getVolumeBasedFee(prizePool);

  // Game-specific adjustment
  const gameAdjustment = this.gameSpecificFees.get(gameType) || 0;
  feePercent += gameAdjustment;

  // Promotional override (e.g., launch discount)
  const promo = await this.getActivePromotion(eventType);
  if (promo) {
    feePercent = Math.max(feePercent - promo.discountPercent, 1.0);
  }
}
```

Business Rules: Fee Examples

Platform Fee Calculation Table

Pool Size	Base Fee	Volume Discount	Promo	Final Fee	Platform Revenue
2 SOL	5.0%	None	None	5.0%	0.10 SOL
50 SOL	5.0%	None	-1% launch	4.0%	2.00 SOL
200 SOL	4.0%	-1% (>100)	None	4.0%	8.00 SOL
500 SOL	4.0%	-1% (>100)	-1% launch	3.0%	15.00 SOL
2000 SOL	3.0%	-2% (>1000)	None	3.0%	60.00 SOL

```
// Fee breakdown for transparency
interface FeeBreakdown {
  totalPool: string;
  baseFeePercent: number;
  volumeDiscount: number;
  promotionalDiscount: number;
  finalFeePercent: number;
  platformFee: string;
  distributableAmount: string;
```

Resources

Documentation

- [Solana Web3.js Docs](#)
- [SPL Token Program](#)
- [MPC Cryptography](#)

Code References

- `/src/modules/esports/` - Full implementation
- `/docs/diagrams/16-esports-matchmaking.md` - Architecture docs
- `/src/modules/mpc/` - MPC wallet integration

Tools

- Anchor Framework for Solana programs

? Questions?

Key Takeaways

1. **MPC wallets** provide security without UX friction
2. **Escrow accounts** ensure trustless prize distribution
3. **Atomic state machines** prevent fund loss
4. **Platform co-signing** enables fraud prevention
5. **Full audit trails** support dispute resolution
6. **Flexible prize strategies** (Winner Takes All, Top 3, MVP) support diverse use cases
7. **Risk-based filtering** helps players find matches matching their preferences



Thank You!

Contact & Resources

-  Project Repository: github.com/psavelis/solana-svm-study
-  Documentation: </docs/diagrams/16-esports-matchmaking.md>
-  Tests: /src/modules/esports/__tests__/

 **Let's Build!**

Production-Ready Esports on Solana